

La anatomía de la NN

curso

Deep Learning



Mauricio A. Valle

Prof. M.A.Valle

E-MAIL: mauricio.valle.b@uai.cl

[illegible]

En el siguiente link, se encuentra una demostración de clasificación de noticias faltas relativas al COVID-19.

Part2: <https://towardsdatascience.com/fake-news-classifier-to-tackle-covid-19-disinformation-ii-116ed2eb44e4>

De todas formas el desafío consiste en utilizar la misma base de datos que se puede descargar aquí:

Utilizar el mismo preprocesamiento, pero intentar diseñar y entrenar una NN que permita predecir el label del texto es FAKE o no.

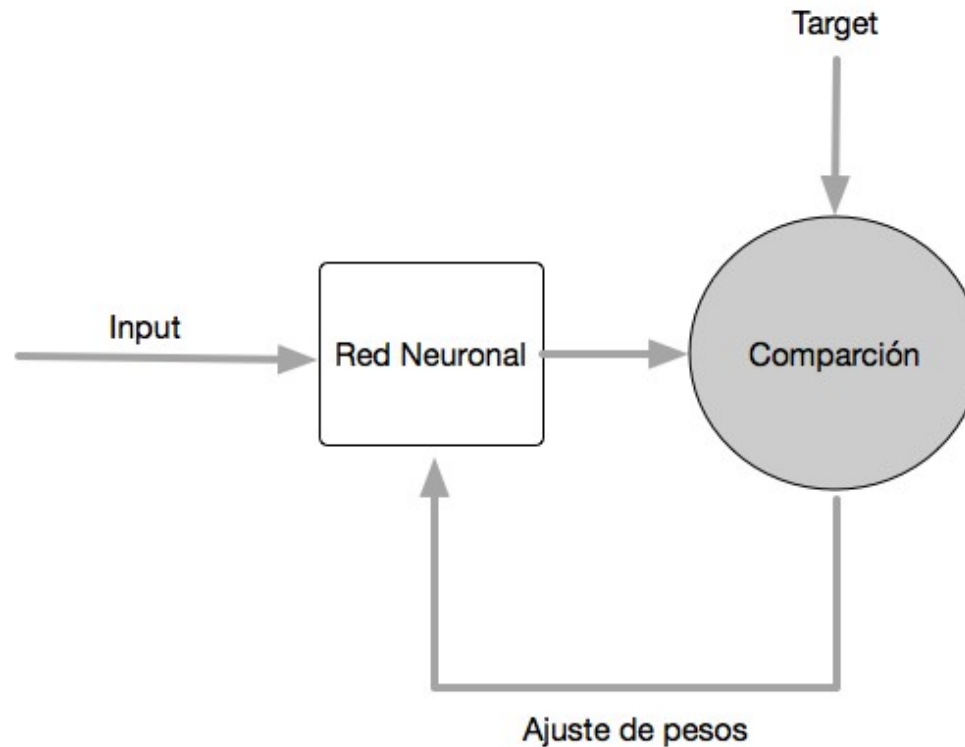
<https://github.com/shaunak09vb/Fake-News-Detection-for-Covid19-Heroku>

Modeling the basic structure

Tenemos 4 componentes básicos de una red neuronal:

- Layers (o capas) que son parte del modelo de la red
- Input data y con su correspondiente variable clase.
- Loss function (o función de pérdida) que define el feedback para el aprendizaje.
- El optimizador, que determina de qué forma se produce el aprendizaje.
- Métrica para monitorear el proceso de entrenamiento y de validación (testing). Por ejemplo el accuracy (fracción de instancias que son clasificadas correctamente).

Modeling the basic structure



La idea del aprendizaje con un perceptrón.

Modeling the basic structure

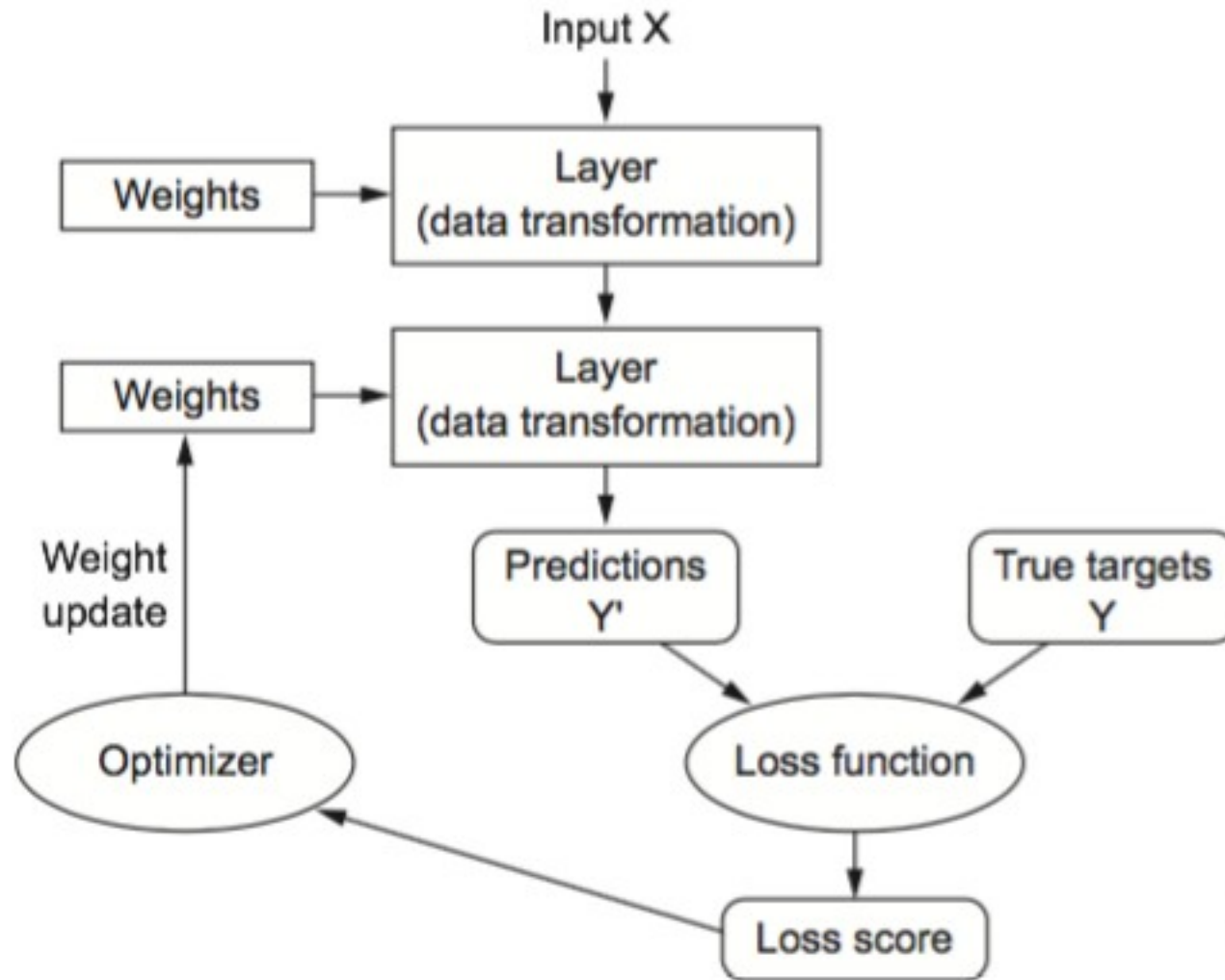
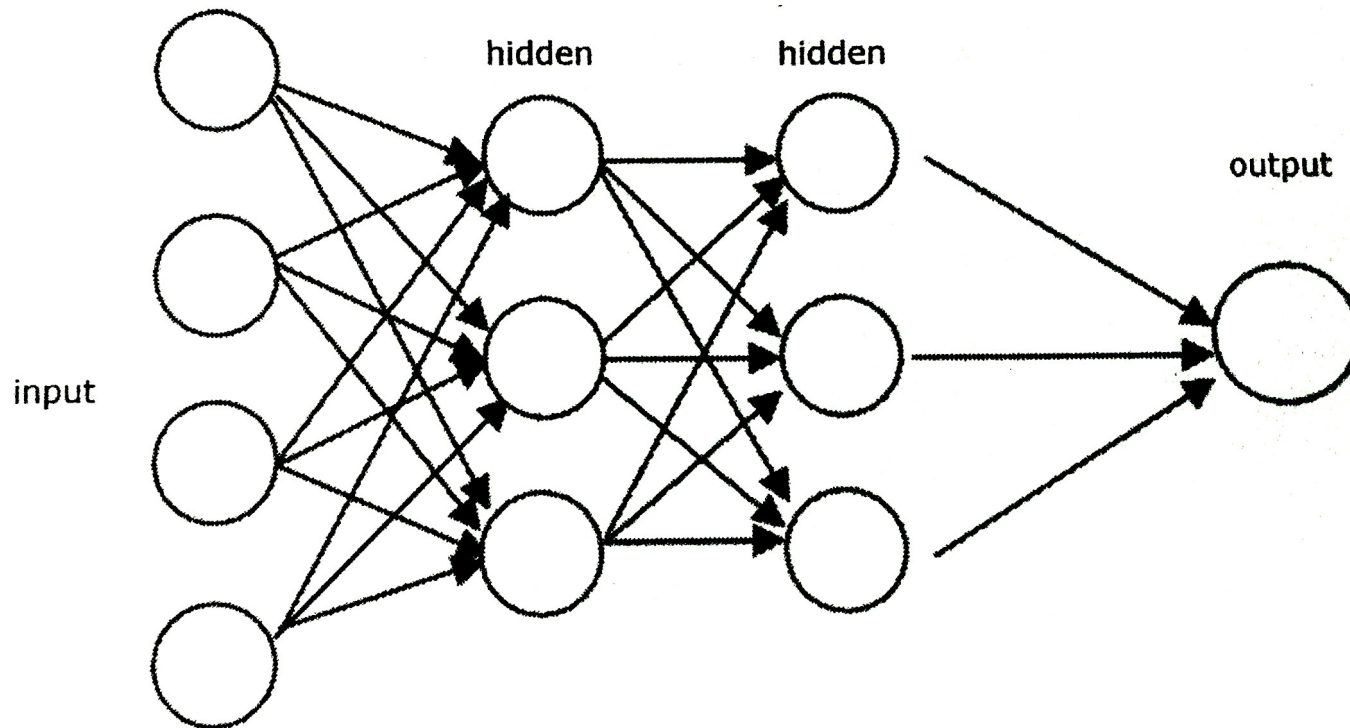


Fig. 3.1 pag. 58 "Deep Learning with Python"

Modeling the basic structure

El flujo de las señales en una red neuronal puede ir en una sólo dirección o en recurrencia. En el primer caso, se trata de una NN feed-forward, dado que las señales comienzan por la capa de entrada, y luego de ser procesadas, se destinan a la siguiente capa hasta llegar a la salida.

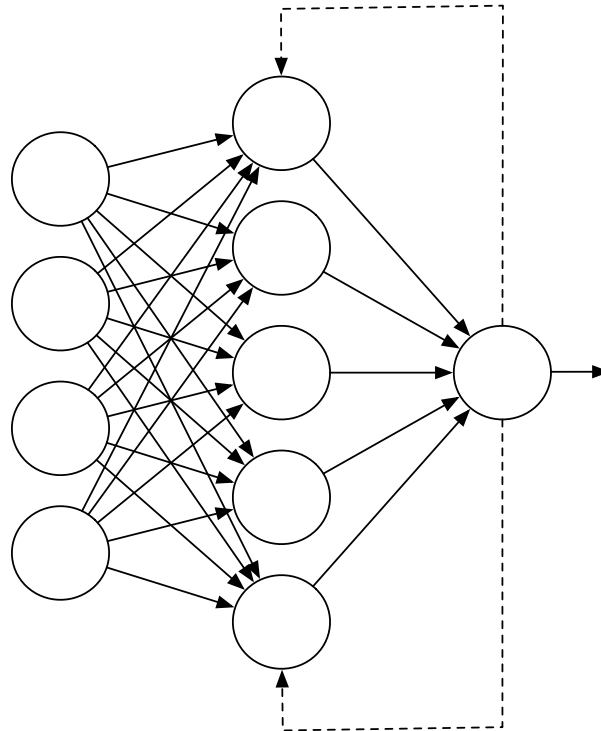
Los MLP (multi layer perceptron) es un buen ejemplo de esto:



Modeling the b

Si la NN posee recurrente
neurona o a una capa que
una feedback NN.

El feedback es importante en
recognition y series de tiempo



Layers o capas

Las NNs se construyen sobre la base de capas: módulos de procesamiento de datos que hacen las veces de “filtro”.

Los datos entran de cierta forma, y salen de otra. Las capas extraen **representaciones** de los datos, con la idea que tengan sentido para el problema en particular.

Es como un proceso de destilación de un licor. Cada capa (proceso de destilación) puesta en forma de secuencia, va extrayendo representaciones de los datos de entrada.

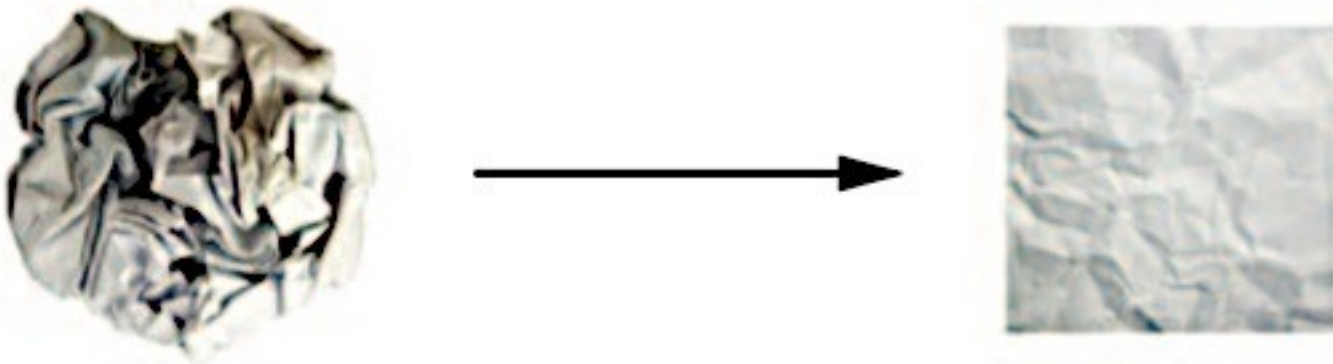


Fig. 2.9 pag. 46 “Deep Learning with Python”

Layers o capas

Los pesos de las capas son actualizadas (learned) con **stochastic gradient descent**.

- Para tensores 2D, utilizamos **densely connected layers** o **fully connected layers** (Dense en Keras).
- Para tensores 3D, utilizamos **recurrent layers**, tales como LSTM long short term memory (imágenes y series de tiempo).

Layers o capas

La **compatibilidad entre layers** (capas) se refiere a que cada capa sólo acepta tensores de entrada de cierta forma y entregará un tensor de salida de otra forma.

Por ejemplo:

```
from keras import layers  
layer = layers.Dense(32, input_shape=(784,))
```



A dense layer with 32 output units

Aquí creamos una capa que solo acepta tensor 2D con 784 atributos (features), y devuelve un tensor con 32, es decir, una transformación de 784 dimensiones a 32.

Layers o capas

Luego, esta capa se puede conectar a otra, que espera 32 vectores como entrada.
Por ejemplo:

```
from keras import models
from keras import layers

model = models.Sequential()
model.add(layers.Dense(32, input_shape=(784,)))
model.add(layers.Dense(32))
```

Fijarse, que la 2da capa no recibe argumento de `input_shape`. En vez de eso, automáticamente, Keras infiere que el input a esta 2da capa es el output de la primera.

Layers o capas

¿qué hacemos en realidad con esto de estar poniendo capas en una NN?

Layers o capas

¿qué hacemos en realidad con esto de estar poniendo capas en una NN?

Desde un punto de vista topológico, la NN es un **grafo acíclico**.

La forma más común de NN es el stacking de capas, que mapean las entradas en salidas.

Pero.... En realidad..

Layers o capas

¿qué hacemos en realidad con esto de estar poniendo capas en una NN?

Desde un punto de vista topológico, la NN es un **grafo acíclico**.

La forma más común de NN es el stacking de capas, que mapean las entradas en salidas.

Pero.... En realidad..

La topología de la NN define el **espacio de hipótesis (boundary decision)**.

La NN intenta buscar una representación útil al espacio de entrada, dentro de un campo de posibilidades, dada la estructura de la red y de la señal de error.

Al seleccionar una topología específica (capas), estamos restringiendo este **espacio de posibilidades** a una serie específica de operaciones tensoriales.

Lo que hacemos, es buscar un buen conjunto de valores para los pesos de las neuronas involucradas en estas operaciones de multiplicación de vectores.

Loss function y Optimizers (el aprendizaje)

Una vez que la arquitectura de la red (capas) está definida, es necesario seleccionar dos cosas:

- Loss function (función objetivo): Lo que se minimizará durante el entretamiento.
- Optimizer: determina cómo se van a actualizar los pesos basado en el función objetivo. Se trata en esencia del **stochastic gradient descent (SGC)**.

Loss function y Optimizers (el aprendizaje)

Loss function:
(advertencia)

Es importante seleccionar la función objetivo correcta para el problema es cuestión. La máquina siempre hará todo lo posible por minimizar la pérdida (loss).

Si el objetivo es inadecuado o no es apropiado para el problema, los resultados pueden tener efectos colaterales no intencionados.

Afortunadamente, ya sabemos que para problemas genéricos, cuáles son las funciones pertinentes. Por ejemplo

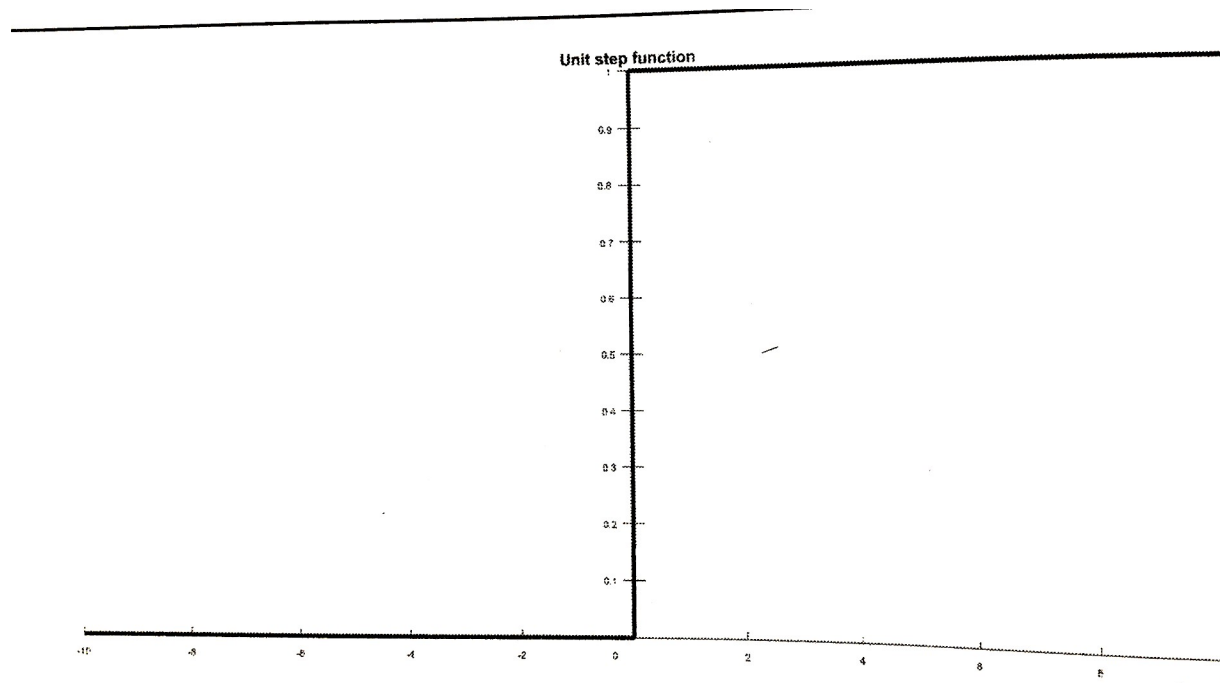
- Para clasificación binaria, usamos **crossentropy**
- Para clasificación multiclass, usamos **categorical crossentropy**
- **Mean squared error** para problemas de regresión,

Loss function y Optimizers (el aprendizaje)

Funciones de activación

Recordemos que la función de activación es una función matemática que convierte una entrada en una salida para una neurona.

Por ejemplo la UNIT STEP ACTIVATION FUNCTION:

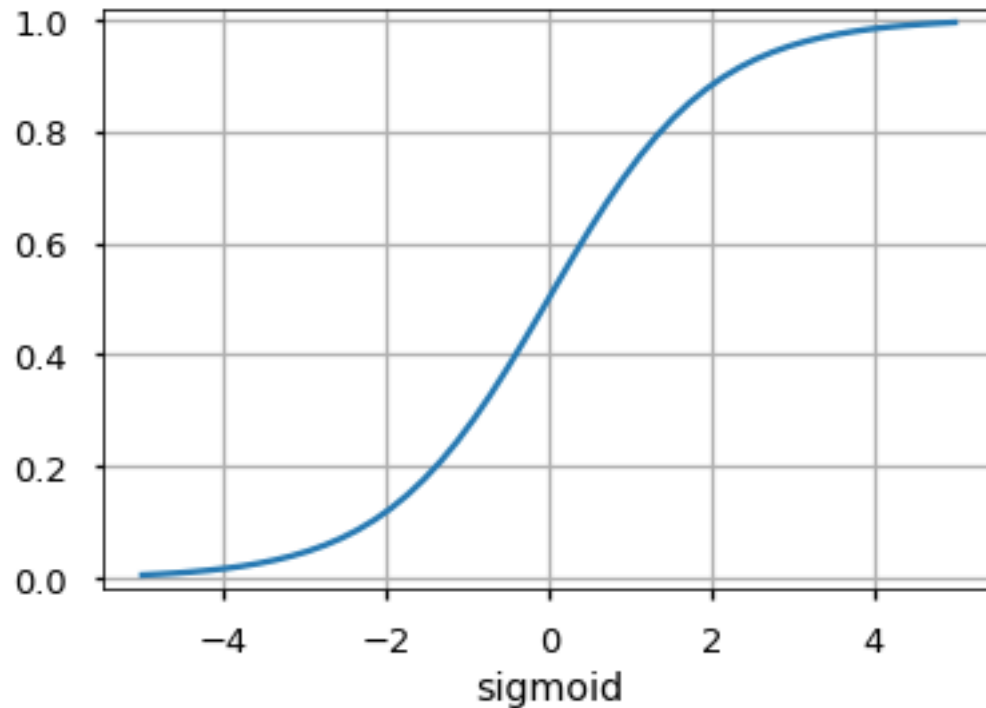


El rango es entre 0 y 1 y la salida es binaria. Útil para clasificaciones binarias.

Loss function y Optimizers (el aprendizaje)

Funciones de activación

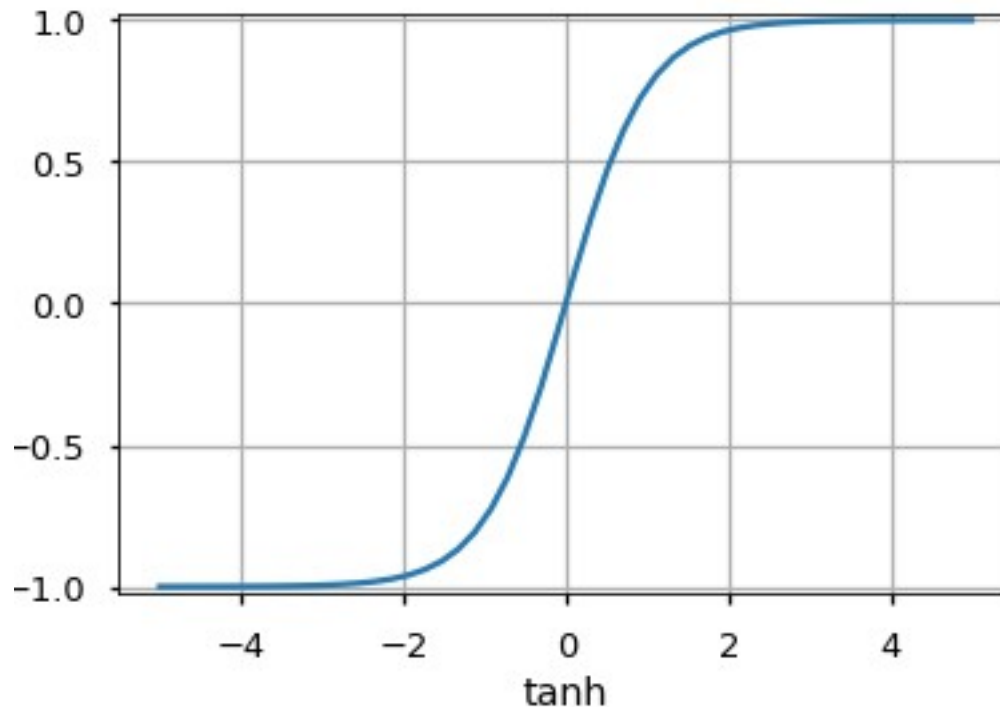
SIGMOIDE: tiene forma de S. Tiene la ventaja sobre la unit step, que es continua y diferenciable. La salida está entre 0 y 1 y es el modelo logístico por naturaleza.



Loss function y Optimizers (el aprendizaje)

Funciones de activación

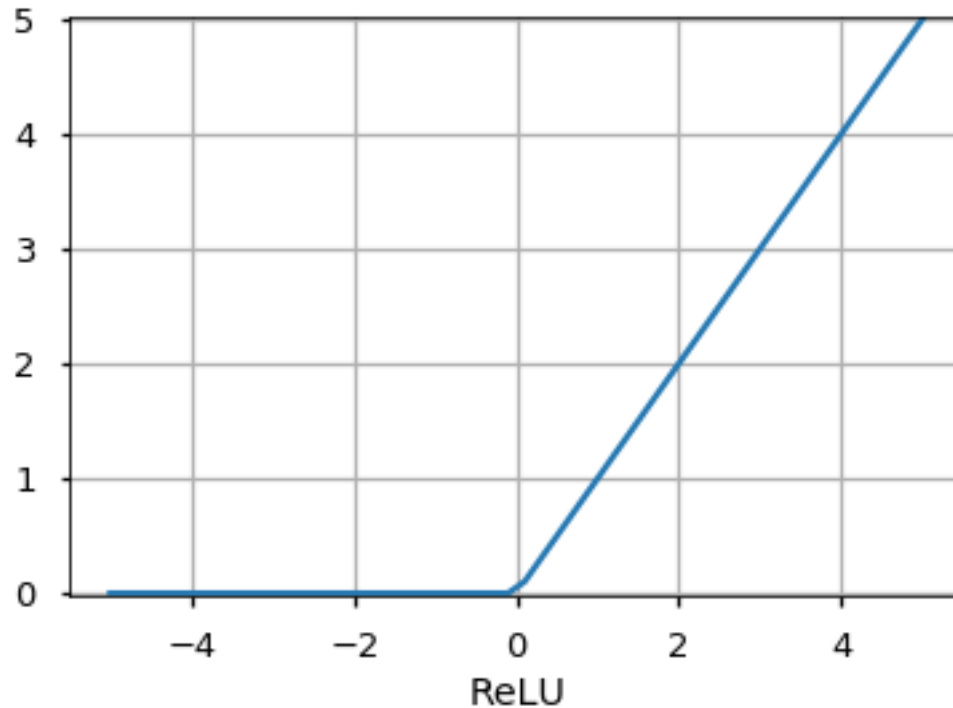
TANGENTE HIPERBOLICA: Es similar a la sigmoide, con la diferencia es que está escalada. Esta entre -1 y +1. El grandiente aquí es más pronunciado que el de la sigmoide.



Loss function y Optimizers (el aprendizaje)

Funciones de activación

RECTIFIED LINEAR UNIT (RELU): Es la función más popular. El rango va de 0 a infinito. Tiene aplicaciones en computer vision y Deep neural nets.



Loss function y Optimizers (el aprendizaje)

Funciones de activación - SOFTMAX

SOFTMAX: Se utiliza por lo general en capa de salida para normalizar la salida de una red en probabilidades para problemas de clasificación multiclass.

La salida es un “vector” de probabilidades, matemáticamente es:

$$S(y)_i = \frac{\exp(y_i)}{\sum_{j=1}^n \exp(y_j)}$$

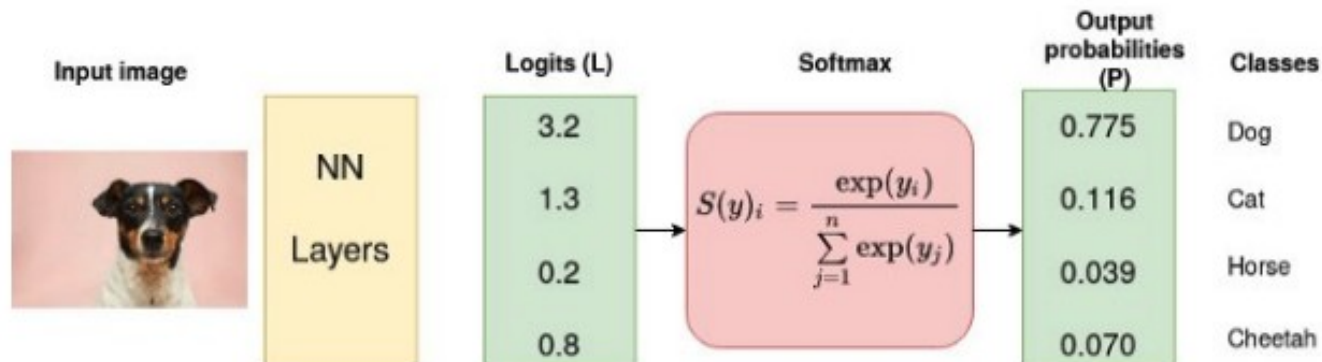
donde y es el vector de entrada de n elementos para cada una de las n clases. Y_i es el i -ésimo valor del vector y .

El numerador es simplemente para normalizar la cantidad y hacer que las probabilidades sumen 1.

Loss function y Optimizers (el aprendizaje)

Funciones de activación - SOFTMAX

Supongamos un problema en que queremos clasificar si una imagen es un perro, gato, caballo o mono. La última capa de la Deep NN resulta en un vector y de salida. Este vector y entra al nodo con función de activación SOFTMAX cuyo resultado son las probabilidades para cada clase.



Fuente: https://unsplash.com/s/photos/dog?utm_source=unsplash&utm_medium=referral&utm_content=creditCopyText

Loss function y Optimizers (el aprendizaje)

Cada neurona transforma la entrada en un output con la función de activación.
En Keras:

```
output = relu(dot(W, input) + b)
```

W y b son los tensores de pesos sinápticos de la la capa (de todas las neuronas que conforman la capa). Son los parámetros (*kernels* y *bias*) los que se van actualizando como resultado del aprendizaje.

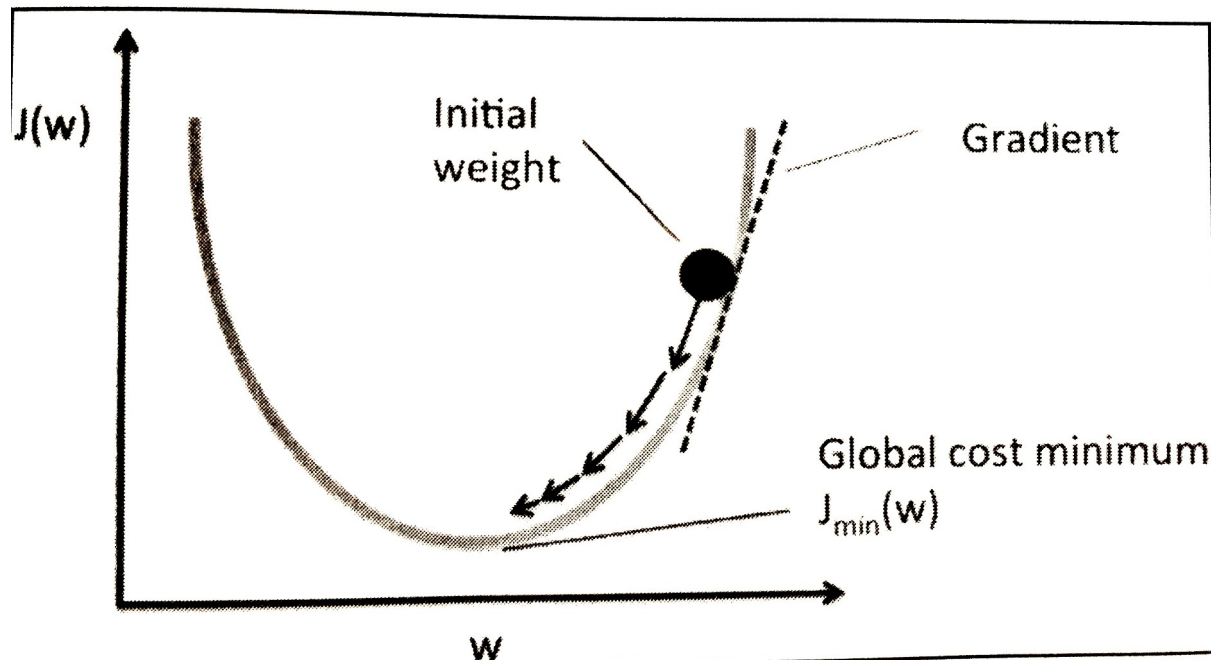
Inicialmente W y b son valores aleatorios (*random initialization*). Luego comienza el **training loop**:

1. Seleccionar un lote de muestras de entrenamiento x y sus respectivas clases y.
2. Correr la red sobre x (forward pass) para obtener y_pred.
3. Computar el loss de la red sobre el lote, y medir la diferencia entre y_pred e y.
4. Actualizar los pesos de red para reducir un poco el loss en este lote.

Loss function y Optimizers (el aprendizaje)

Gradient descent: la idea

El gradient descent es una aproximación iterativa para la corrección del error en un aprendizaje. Para las NN con backpropagation, es el **proceso de corregir los pesos proporcional a la derivada de la función de activación**.



La idea de tener el gradient descent en cada paso, es encontrar el mínimo loss global. Stochastics Gradient Descent significa que vamos tomando una muestra de a una y hacemos los cálculos del gradiente.

Loss function y Optimizers (el aprendizaje)

Gradient descent: la idea

Imagine un vector de entrada $\mathbf{x}$, una matriz $\mathbf{W}$ de pesos, y el vector de clases $\mathbf{y}$, y una función de pérdida loss.

Podemos utilizar $\mathbf{W}$ para computar y_{pred} , computar loss (el grado de unfit entre y vs y_{pred}):

```
y_pred = dot(W, x)
loss_value = loss(y_pred, y)
```

Loss function y Optimizers (el aprendizaje)

Gradient descent: la idea

Imagine un vector de entrada $\mathbf{x}$, una matriz $\mathbf{W}$ de pesos, y el vector de clases $\mathbf{y}$, y una función de pérdida loss.

Podemos utilizar $\mathbf{W}$ para computar y_{pred} , computar loss (el grado de unfit entre y vs y_{pred}):

```
y_pred = dot(W, x)
loss_value = loss(y_pred, y)
```

Si dejamos x e y congelados, podemos calcular el valor de mismatch:

```
loss_value = f(W)
```

Loss function y Optimizers (el aprendizaje)

Gradient descent: la idea

Imagine un vector de entrada $\mathbf{x}$, una matriz $\mathbf{W}$ de pesos, y el vector de clases $\mathbf{y}$, y una función de pérdida loss .

Podemos utilizar $\mathbf{W}$ para computar $\mathbf{y_pred}$, computar loss (el grado de unfit entre $\mathbf{y}$ vs $\mathbf{y_pred}$):

```
 $\mathbf{y\_pred} = \text{dot}(\mathbf{W}, \mathbf{x})$   
 $\text{loss\_value} = \text{loss}(\mathbf{y\_pred}, \mathbf{y})$ 
```

Si dejamos $\mathbf{x}$ e $\mathbf{y}$ congelados, podemos calcular el valor de mismatch:

```
 $\text{loss\_value} = f(\mathbf{W})$ 
```

Supongamos que el valor de $\mathbf{W}$ es $\mathbf{W}_0$. La derivada de f (activation function) en $\mathbf{W}_0$ es el tensor $\text{gradient}(f)(\mathbf{W}_0)$, en donde cada coeficiente $\text{gradient}(f)(\mathbf{W}_0)[i, j]$ indica la dirección y magnitud de cambio en loss_value que se observa al modificar $(\mathbf{W}_0)[i, j]$. Ese tensor $\text{gradient}(f)(\mathbf{W}_0)$ es el gradiente de la función $f(\mathbf{W}) = \text{loss_value}$ en $\mathbf{W}_0$.

Loss function y Optimizers (el aprendizaje)

Gradient descent: la idea

Pensemos que el $\text{gradient}(f)(W_0)$ se interpreta como la pendiente de la curva de f . Así $\text{gradient}(f)(W_0)$ es el tensor que describe la curvatura de $f(w)$ alrededor de W_0 .

Por esta razón, para una función $f(x)$, podemos reducir el valor de $f(x)$ moviendo x un poco en la dirección opuesta a la dirección de la derivada.

Con una función $f(W)$ de un tensor, podemos reducir $f(W)$ moviendo W en la dirección opuesta a la dirección del gradiente, por ejemplo:

$$W_1 = W_0 - \text{step} * \text{gradient}(f)(W_0)$$

donde step es sólo una constante.

Esto significa que no movemos en contra de la curvatura, que equivale a decir que nos acercamos a una parte más plana o descendemos en la curva.

Loss function y Optimizers (el aprendizaje)

Stochastic Gradient descent: la idea

La idea es MINIMIZAR el error (loss). Esto equivale a decir a encontrar los valores de W tal que el gradiente sea cero, es decir,

$$\text{gradient}(f)(W_0) = 0$$

Esto se puede hacer analíticamente para $N = 2$ o 3 , pero en un problema de una red neuronal, el número de parámetros es de miles e incluso millones!

What!?!?! Qué podemos hacer??????

Loss function y Optimizers (el aprendizaje)

Stochastic Gradient descent: mini batch

Podemos utilizar un algoritmo, en que los pesos van cambiando poco a poco basado en el valor actual de los valores de los atributos:

1. Seleccionar un lote de muestras de x y de sus correspondientes clases y .
2. Correr la red sobre x y obtener y_{pred} .
3. Computar $loss$: mismatch entre y vs y_{pred} .
4. Computar el gradiente de $loss$ en relación a los parámetros de la red (backward pass)
5. Mover los parámetros en la dirección opuesta del gradiente
 $W -= step * gradient$

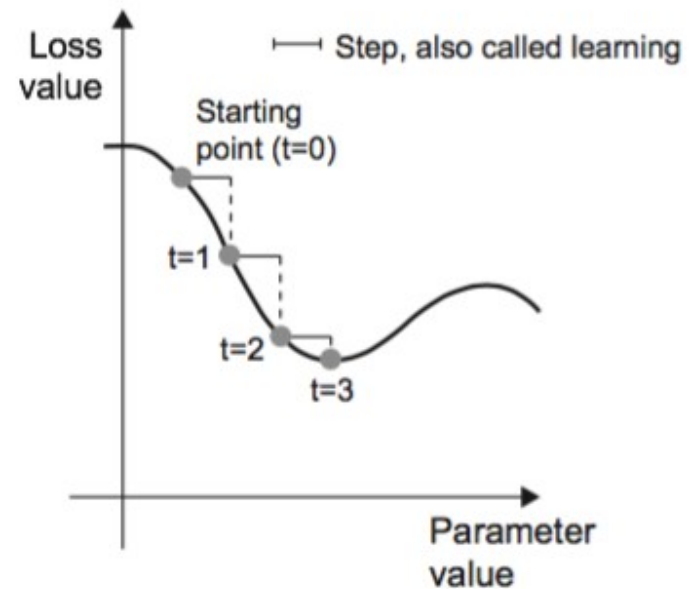


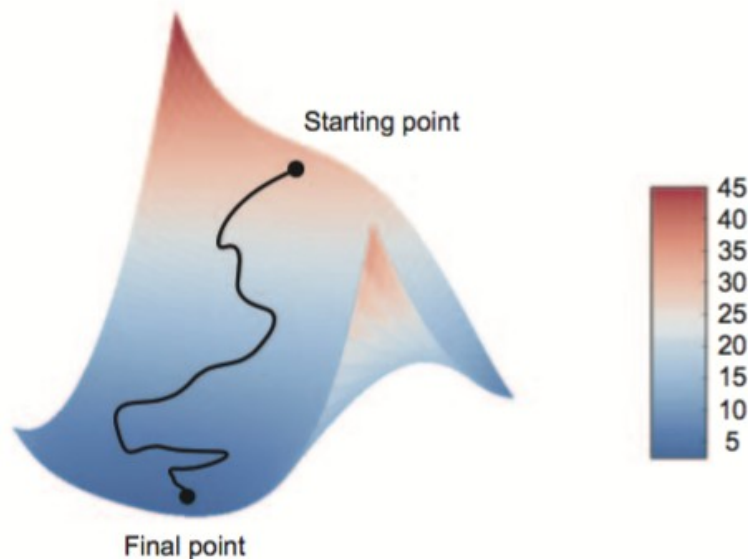
Fig. 2.11 pag. 49 “Deep Learning with Python”

Loss function y Optimizers (el aprendizaje)

Stochastic Gradient descent: mini batch

Con respecto al factor **step** = LEARNING RATE : si elegimos uno muy pequeño, el descenso por la curva será muy lento y necesitaremos muchas iteraciones. Si es muy grande, vamos a ir fluctuando por la curva en forma errática.

El proceso es estocástico, porque elegimos lotes (batch) de datos se seleccionan en forma aleatoria.



Podemos visualizar el descenso de gradiente en una 2D loss surface, en vez de 1D.

Obviamente en la realidad, la red neuronal hace el trabajo de aprendizaje sobre miles incluso cientos de miles de dimensiones.

Fig. 2.12 pag. 50 "Deep Learning with Python"

Loss function y Optimizers (el aprendizaje)

Stochastic Gradient descent: mini batch

Tenemos múltiples variaciones del SGD en que el proceso toma en cuenta la actualización de los pesos previos al computar la siguiente actualización, y no sólo los valores actuales del gradiente.

Estas versiones de SGD se denominan OPTIMIZERS (por ejemplo, Adagrad, RMSProp, etc) que consideran el momentum = monitorean la velocidad de convergencia y mínimos locales.

Si el Learning rate es muy pequeño, podemos quedar atascado en un mínimo local.

Lo que podemos hacer utilizar MOMENTUM = update w no sólo con el valor actual del gradiente, sino considerando el valor previo de W .

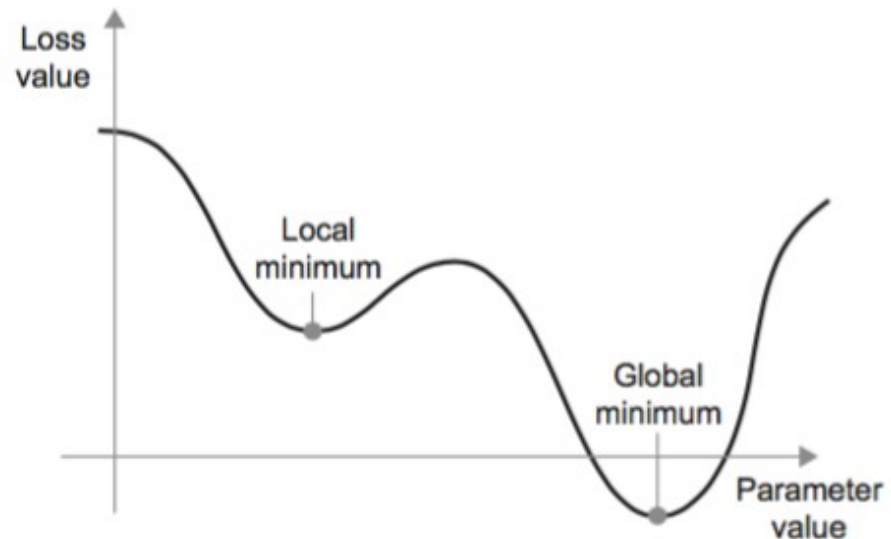


Fig. 2.13 pag. 51 "Deep Learning with Python"

Prácticas de NN en Keras

A jugar! (nn_cancer_datasetbinaryclass_example.ipynb)

Volvamos nuevamente a Jupyter Lab para implementar nuestra red neuronal para detección de cancer de pecho.

Descripción de los datos:

<https://www.kaggle.com/uciml/breast-cancer-wisconsin-data>

Features are computed from a digitized image of a fine needle aspirate (FNA) of a breast mass. They describe characteristics of the cell nuclei present in the image.

n the 3-dimensional space is that described in: [K. P. Bennett and O. L. Mangasarian: "Robust Linear Programming Discrimination of Two Linearly Inseparable Sets", Optimization Methods and Software 1, 1992, 23-34].

This database is also available through the UW CS ftp server:

ftp ftp.cs.wisc.edu

cd math-prog/cpo-dataset/machine-learn/WDBC/

Also can be found on UCI Machine Learning Repository:

<https://archive.ics.uci.edu/ml/datasets/Breast+Cancer+Wisconsin+%28Diagnostic%29>

A jugar! (nn_cancer_datasetbinaryclass_example.ipynb)

Volvamos nuevamente a Jupyter Lab para implementar nuestra red neuronal para detección de cancer de pecho.

Attribute Information:

- 1) ID number
- 2) Diagnosis (M = malignant, B = benign
3-32)

Ten real-valued features are computed for each cell nucleus:

- a) radius (mean of distances from center to points on the perimeter)
- b) texture (standard deviation of gray-scale values)
- c) perimeter
- d) area
- e) smoothness (local variation in radius lengths)
- f) compactness ($\text{perimeter}^2 / \text{area} - 1.0$)
- g) concavity (severity of concave portions of the contour)
- h) concave points (number of concave portions of the contour)
- i) symmetry
- j) fractal dimension ("coastline approximation" - 1)

The mean, standard error and "worst" or largest (mean of the three largest values) of these features were computed for each image, resulting in 30 features. For instance, field 3 is Mean Radius, field 13 is Radius SE, field 23 is Worst Radius.

All feature values are recoded with four significant digits.

Missing attribute values: none

Class distribution: 357 benign, 212 malignant

A jugar! (nn_fashion_dataset_example.ipynb)

Volvamos nuevamente a Jupyter Lab para implementar nuestra red neuronal con aplicación de reconocimiento de imágenes!

Descripción de los datos:

<https://www.kaggle.com/zalando-research/fashionmnist>

Context

Fashion-MNIST is a dataset of Zalando's article images—consisting of a training set of 60,000 examples and a test set of 10,000 examples. Each example is a 28×28 grayscale image, associated with a label from 10 classes. Zalando intends Fashion-MNIST to serve as a direct drop-in replacement for the original MNIST dataset for benchmarking machine learning algorithms. It shares the same image size and structure of training and testing splits.

The original MNIST dataset contains a lot of handwritten digits. Members of the AI/ML/Data Science community love this dataset and use it as a benchmark to validate their algorithms. In fact, MNIST is often the first dataset researchers try. "If it doesn't work on MNIST, it won't work at all", they said. "Well, if it does work on MNIST, it may still fail on others."

Zalando seeks to replace the original MNIST dataset

Content

Each image is 28 pixels in height and 28 pixels in width, for a total of 784 pixels in total. Each pixel has a single pixel-value associated with it, indicating the lightness or darkness of that pixel, with higher numbers meaning darker. This pixel-value is an integer between 0 and 255. The training and test data sets have 785 columns. The first column consists of the class labels (see above), and represents the article of clothing. The rest of the columns contain the pixel-values of the associated image.

- To locate a pixel on the image, suppose that we have decomposed x as $x = i * 28 + j$, where i and j are integers between 0 and 27. The pixel is located on row i and column j of a 28×28 matrix.
- For example, pixel31 indicates the pixel that is in the fourth column from the left, and the second row from the top, as in the ascii-diagram below.

A jugar!

Volvamos nuevamente a Jupyter Lab para implementar nuestra primera red neuronal con aplicación de reconocimiento de imágenes!

Descripción de los datos:

<https://www.kaggle.com/zalando-research/fashionmnist>

Labels

Each training and test example is assigned to one of the following labels:

- 0 T-shirt/top
- 1 Trouser
- 2 Pullover
- 3 Dress
- 4 Coat
- 5 Sandal
- 6 Shirt
- 7 Sneaker
- 8 Bag
- 9 Ankle boot

TL;DR

- Each row is a separate image
- Column 1 is the class label.
- Remaining columns are pixel numbers (784 total).
- Each value is the darkness of the pixel (1 to 255)

A jugar!

La arquitectura involucra una NN con 4 capas.

La primera con las 784 atributos de brillo de pixel.

La segunda escondida con 64 neuronas,

La tercera escondida también con 64 neuronas.

La cuarta capa de salida con 10 neuronas para cada una de las 10 posibles clases.

Como es tradicional en problemas multiclass, en la última capa, se aplica una función de activación SOFTMAX:

$$P(y = j \mid \mathbf{x}) = \frac{e^{\mathbf{x}^T \mathbf{w}_j}}{\sum_{k=1}^K e^{\mathbf{x}^T \mathbf{w}_k}}$$

Lo que hace es simplemente, para cada una de las neuronas de salida (las 10 en este caso) es calcular la probabilidad de que la instancia pertenezca a la j-ésima clase (de las $K = 10$ posibles clases en este ejemplo)

A jugar3! (nn_handnumbers_datasets_example.ipynb)

Trabajo en clase:

Construir NN para identificar números del 0 al 9 escritos a mano.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# arquitectura
model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(dim_data,)))
model.add(Dense(512, activation='relu'))
model.add(Dense(classes_num, activation='softmax'))

# configuracion
model.compile(optimizer='rmsprop', loss='categorical_crossentropy', metrics=['accuracy'])

# train the model
history = model.fit(train_data, train_labels_one_hot, batch_size=256, epochs=20,
                    verbose=1, validation_data=(test_data, test_labels_one_hot))

# Evaluation
test_loss, test_acc] = model.evaluate(test_data, test_labels_one_hot)
print("Evaluation result on Test Data : Loss = {},
      accuracy = {}".format(test_loss, test_acc))
```

A jugar4! (nn_airbnb_datasets_example.ipynb)

Trabajo en clase:

Construir NN para para un regresión que nos permita predecir el precio de un servicio AIRBNB.

El dataset lo podemos encontrar en:

<https://www.kaggle.com/stevezhenghp/airbnb-price-prediction>

Aquí no tenemos variable clase, tenemos una variable continua que es el log natural del precio del servicio con una serie de atributos.

Este ejemplo nos servirá también para ver la importancia del preprocesamiento de los datos antes de iniciar el entrenamiento.

A jugar 5!

Volvamos nuevamente a Jupyter Lab para implementar una red neuronal.

Para esto vamos a utilizar una dataset llamado IMDB: unset de 50,000 revisiones / comentarios positivos y negativos de películas del Internet Movie Database (<https://www.imdb.com/>).

Vamos a dividir la base de datos en dos partes: 25,000 para el entrenamiento o otras 25,000 para el testeo (cada una con 50% de comentarios + y 50% con comentarios -)

Esta división lo hacemos como medida para prevenir el **overfitting**.

Afortunadamente, la base de datos ya viene incorporada en KERAS 😊